

C++ Standard Library Issues to be moved in Jacksonville

Doc. no. P0165R1

Date: Revised 2016-02-12 at 23:02:13 UTC

Project: Programming Language C++

Reply to: Marshall Clow <lwgchair@gmail.com>

Ready Issues

[2192](#). Validity and return type of `std::abs(0u)` is unclear

Section: 26.8 [c.math] **Status:** [Tentatively Ready](#) **Submitter:** Daniel Krügler **Opened:** 2012-10-02 **Last modified:** 2015-11-04

Priority: 2

View other [active issues](#) in [c.math].

View all other [issues](#) in [c.math].

View all issues with [Tentatively Ready](#) status.

Discussion:

In C++03 the following two programs are invalid:

```
A.      #include <cmath>

        int main() {
            std::abs(0u);
        }

B.      #include <cstdlib>

        int main() {
            std::abs(0u);
        }
```

because none of the `std::abs()` overloads is a best match.

In C++11 the additional "sufficient overload" rule from 26.8 [c.math] p11 (see also LWG [2086](#)) can be read to be applicable to the `std::abs()` overloads as well, which can lead to the following possible conclusions:

1. The program

```
#include <type_traits>
#include <cmath>

static_assert(std::is_same<decltype(std::abs(0u)), double>(), "Oops");

int main() {
    std::abs(0u); // Calls std::abs(double)
}
```

is required to be well-formed, because of sub-bullet 2 ("[..] or an integer type [..]") of 26.8 [c.math] p11 (Note that the current resolution of LWG [2086](#) doesn't fix this problem).

2. Any translation unit including both `<cmath>` and `<cstdlib>` might be ill-formed because of two conflicting requirements for the return type of the overload `std::abs(int)`.

It seems to me that at least the second outcome is not intended, personally I think that both are unfortunate: In contrast to all other floating-point functions explicitly listed in sub-clause 26.8 [c.math], the `abs` overloads have a special and well-defined meaning for signed integers and thus have explicit overloads returning a signed integral type. I also believe that there is no problem accepting that `std::fabs(0u)` is well-defined with return type `double`, because the leading 'f' clearly signals that we have a floating point function here. But the expected return type of `std::abs(0u)` seems less than clear to me. A very reasonable answer could be that this has the same type as its argument type, alternatively it could be a reasonably chosen signed integer

type, or a floating point type. It should also be noted, that the corresponding "generic type function" rule set from C99/C1x in 7.25 p2+3 is restricted to the floating-point functions from `<math.h>` and `<complex.h>`, so cannot be applied to the `abs` functions (but to the `fabs` functions!).

Selecting a signed integer return type for unsigned input values can also be problematic: The directly corresponding signed integer type would give half of the possible argument values an implementation-defined result value. Choosing the first signed integer value that can represent all positive values would solve this problem for `unsigned int`, but there would be no clear answer for the input type `std::uintmax_t`.

Based on this it seems to me that the C++03 state in regard to unsigned integer values was the better situation, alerting the user that this code is ambiguous at the moment (This might be change with different core-language rules as described in N3387).

[2013-04-20, Bristol]

Resolution: leave as new and bring it back in Chicago.

[2013-09 Chicago]

This issue also relates to LWG [2294](#)

STL: these two issues should be bundled

Stefanus: do what Pete says, and add overloads for unsigned to return directly

STL: agree Consensus that this is an issue

Walter: motion to move to Open

STL: no wording for [2294](#)

Stefanus: move to open and note the 2 issues are related and should be moved together

Stefanus: add and define unsigned versions of `abs()`

[2014-02-03 Howard comments]

Defining `abs()` for unsigned integers is a bad idea. Doing so would turn compile time errors into run time errors, especially in C++ where we have templates, and the types involved are not always apparent to the programmer at design time. For example, consider:

```
template <class Int>
Int
analyze(Int x, Int y)
{
    // ...
    if (std::abs(x - y) < threshold)
    {
        // ...
    }
    // ...
}
```

`std::abs(expr)` is often used to ask: Are these two numbers sufficiently close? When the assumption is that the two numbers are signed (either signed integral, or floating point), the logic is sound. But when the same logic is accidentally used with an arithmetic type not capable of representing negative numbers, and especially if unsigned overflow will silently happen, then the logic is no longer correct:

```
auto i = analyze(20u, 21u); // Today a compile time error
// But with abs(unsigned) becomes a run time error
```

This is not idle speculation. Search the net for "abs unsigned" [here](#) or [here](#).

In C++11, `chrono::duration`s and `time_points` are allowed to be based on unsigned integers. Taking the absolute value of the difference of two such `time_points` would be easy to accidentally do (say in code templated on `time_points`), and would certainly be a logic bug, caught at compile time unless we provide the error prone `abs(unsigned)`.

[2015-02, Cologne]

GR: Do we want to make the changes to both `<cmath>` and `<cstdlib>`?

AM: I think so; we should provide consistent overloads.

GR: Then we're imposing restrictions on what users put in the global namespace.

AM: I'm not so worried about that. Users already know not to use C library names.

VV: So what are we going to do about unsigned integers? AM: We will say that they are ill-formed.

AM: Does anyone volunteer to send updated wording to Daniel? GR, can you do it? GR: Sure.

GR: To clarify: we want to make unsigned types ill-formed?

AM: With promotion rank at least unsigned int.

GR: And NL suggests to just list those types.

Conclusion: Merge the resolution into a single issue.

Previous resolution from Daniel [SUPERSEDED]:

This wording is relative to N3376.

1. Change 26.8 [c.math] p11 as indicated:

-11- Moreover, except for the abs functions, there shall be additional overloads sufficient to ensure:

[...]

[2015-03-03, Geoffrey Romer provides improved wording]

In the following I've drafted combined wording to resolve LWG [2192](#) and [2294](#). Note that the first two paragraphs are taken verbatim from the P/R of LWG [2294](#), but the third is newly drafted:

[2015-05-05 Lenexa: Howard to draft updated wording]

[2015-09-11: Howard updated wording]

[2015-10, Kona Saturday afternoon]

HH: abs() for unsigned types is really dangerous. People often use abs(x - y), which would be a disaster.

TK: That's why you need a two-argument abs_diff(x, y), especially for unsigned types.

JW: Lawrence has a proposal for abs_diff in the mailing.

STL: As an alternative to considering promotions, I would just ban all unsigned types, even unsigned char.

STL: Does the PR change any implementation? Is the final paragraph just a consequence?

HH: It's a consequence. It could just be a note.

VV: Ship it as is.

STL: Editorial: capitalize the first letter in the Note.

Move to Tentatively ready.

Proposed resolution:

This wording is relative to N4527.

1. Insert the following new paragraphs after 26.8 [c.math] p7:

-6- In addition to the int versions of certain math functions in <cstdlib>, C++ adds long and long long overloaded versions of these functions, with the same semantics.

-7- The added signatures are:

```
long abs(long); // labs()
long long abs(long long); // llabs()
ldiv_t div(long, long); // ldiv()
lldiv_t div(long long, long long); // lldiv()
```

-?- To avoid ambiguities, C++ also adds the following overloads of abs() to <cstdlib>, with the semantics defined in <cmath>:

```
float abs(float);
double abs(double);
```

```
long double abs(long double);
```

-?- To avoid ambiguities, C++ also adds the following overloads of `abs()` to `<cmath>`, with the semantics defined in `<cstdlib>`:

```
int abs(int);
```

```
long abs(long);
```

```
long long abs(long long);
```

-?- If `abs()` is called with an argument of type `x` for which `is_unsigned<X>::value` is `true` and if `x` cannot be converted to `int` by integral promotion (4.5 [conv.prom]), the program is ill-formed. [Note: arguments that can be promoted to `int` are permitted for compatibility with C. — end note]

2253. [arrays.ts] dynarray should state which container requirements aren't met

Section: 99 [arrays.ts::dynarray.overview] **Status:** [Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2013-04-23 **Last modified:** 2016-02-12

Priority: 0

View all issues with [Ready](#) status.

Discussion:

Addresses: arrays.ts

99 [dynarray.overview] p2 says:

"Unless otherwise specified, all `dynarray` operations have the same requirements and semantics as specified in 99 [container.requirements]."

Some differences from 99 [container.requirements] are not explicitly specified, including at least the lack of a default constructor, copy assignment and `swap` member.

The wording could be similar to 23.3.7.1 [array.overview] which says "An array satisfies all of the requirements of a container and of a reversible container (99 [container.requirements]), except that a default constructed array object is not empty and that `swap` does not have constant complexity."

[2013-09 Chicago:]

Move to Deferred. This feature will ship after C++14 and should be revisited then.

[2014-06-06 pre-Rapperswil]

This issue has been reopened as arrays-ts.

[2014-06-16 Rapperswil]

Move to Ready

[2014/11 Urbana]

Held at Ready status, pending clarification of Arrays TS

Proposed resolution:

1. Add to 99 [dynarray.overview] p2:

-2- A `dynarray` satisfies all of the requirements of a container and of a reversible container (99 [container.requirements]), except for default construction, assignment and `swap`. Unless otherwise specified, all `dynarray` operations have the same requirements and semantics as specified in 99 [container.requirements].

2255. [arrays.ts] dynarray constructor ambiguity

Section: 99 [arrays.ts::dynarray.cons] **Status:** [Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2013-04-23 **Last modified:** 2016-02-12

Priority: 0

View all issues with [Ready](#) status.

Discussion:

Addresses: arrays.ts

These constructors can interact badly::

```
template<class Alloc>
    dynarray(size_type c, const Alloc& alloc);
dynarray(size_type c, const T& v);
```

Unless the second argument is a value of exactly the type τ you will get the first constructor, i.e. all of these will fail to compile:

```
dynarray<long> dlong(1, 1);    // 1 is not long
dynarray<float> dflt(1, 1.0);  // 1.0 is not float
dynarray<int*> dptr(1, nullptr); // nullptr is not int*
dynarray<void*> doh(1, 0);    // 0 is not void*
```

The `nullptr` case is particularly annoying, a user trying to do the right thing by saying `nullptr` instead of `NULL` still gets the wrong result.

The constructor taking an allocator requires that "Alloc shall meet the requirements for an Allocator" but doesn't actually say "shall not participate in overload resolution unless ..."

I believe we have no precedent for using SFINAE to check "the requirements for an Allocator" because it's a pretty complicated set of requirements. We could say it shall not participate in overload resolution if `Alloc` is implicitly convertible to `value_type`.

Alternatively, we could follow the same approach used by other types that can be constructed with an unconstrained allocator type and use `std::allocator_arg_t` as the first argument instead of adding an allocator after the other arguments.

[2013-09 Chicago:]

Move to Deferred. This feature will ship after C++14 and should be revisited then.

[2014-06-06 pre-Rapperswil]

This issue has been reopened as arrays-ts.

[2014-06-16 Rapperswil]

Move to Ready for alternative A

Previous resolution [SUPERSEDED]:

- A. *Either* use the correct way to unambiguously call a constructor taking any type of allocator, i.e. change the constructors to take `dynarray(std::allocator_arg_t, const Alloc&, ...)` by modifying both the synopsis 99 [dynarray.overview] p2 and 99 [dynarray.cons] before p9 like so:

```
template <class Alloc>
    dynarray(allocator_arg_t, const Alloc& a, size_type c, const Alloc& alloc);
template <class Alloc>
    dynarray(allocator_arg_t, const Alloc& a, size_type c, const T& v, const Alloc& alloc);
template <class Alloc>
    dynarray(allocator_arg_t, const Alloc& a, const dynarray& d, const Alloc& alloc);
template <class Alloc>
    dynarray(allocator_arg_t, const Alloc& a, initializer_list<T>, const Alloc& alloc);
```

- B. *or* constrain the problematic constructor by adding a new paragraph to 99 [dynarray.cons]:

```
template <class Alloc>
    dynarray(size_type c, const Alloc& alloc);
template <class Alloc>
    dynarray(size_type c, const T& v, const Alloc& alloc);
template <class Alloc>
    dynarray(const dynarray& d, const Alloc& alloc);
template <class Alloc>
    dynarray(initializer_list<T>, const Alloc& alloc);
```

-9- *Requires:* Alloc shall meet the requirements for an Allocator (17.6.3.5 [allocator.requirements]).

-10- *Effects:* Equivalent to the preceding constructors except that each element is

constructed with uses-allocator construction (20.7.7.2 [allocator.uses.construction]).

[-? Remarks: The first constructor shall not participate in overload resolution unless Alloc is not implicitly convertible to T.

[2014/11 Urbana]

Held at Ready status, pending clarification of Arrays TS

Proposed resolution:

1. Use the correct way to unambiguously call a constructor taking any type of allocator, i.e. change the constructors to take `dynarray(std::allocator_arg_t, const Alloc&, ...)` by modifying both the synopsis 99 [dynarray.overview] p2 and 99 [dynarray.cons] before p9 like so:

```
template <class Alloc>
    dynarray(allocator_arg_t, const Alloc& a, size_type c, const Alloc& alloc);
template <class Alloc>
    dynarray(allocator_arg_t, const Alloc& a, size_type c, const T& v, const Alloc& alloc);
template <class Alloc>
    dynarray(allocator_arg_t, const Alloc& a, const dynarray& d, const Alloc& alloc);
template <class Alloc>
    dynarray(allocator_arg_t, const Alloc& a, initializer_list<T>, const Alloc& alloc);
```

2276. Missing requirement on `std::promise::set_exception`

Section: 30.6 [futures] **Status:** [Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2013-07-30 **Last modified:** 2016-02-12

Priority: Not Prioritized

View all other [issues](#) in [futures].

View all issues with [Ready](#) status.

Discussion:

The standard does not specify the behaviour of this program:

```
#include <future>
#include <cassert>

struct NonTrivial
{
    NonTrivial() : init(true) { }
    ~NonTrivial() { assert(init); }
    bool init;
};

int main()
{
    std::promise<NonTrivial> p;
    auto f = p.get_future();
    p.set_exception(std::exception_ptr());
    f.get();
}
```

The standard doesn't forbid making the state ready with a null `exception_ptr`, so what should `get()` return? There's no stored exception to throw, but it can't return a value because none was initialized.

A careful reading of the standard shows 30.6.4 [futures.state] p8 says "A shared state is *ready* only if it holds a value or an exception ready for retrieval." One can infer from the fact that `set_exception()` makes the state ready that it must store a value or exception, so cannot store "nothing", but that isn't explicit.

The `promise::set_exception()` and `promise::set_exception_at_thread_exit()` members should require `p != nullptr` or should state the type of exception thrown if `p` is null.

[2015-02 Cologne]

Handed over to SG1.

[2015-05 Lenexa, SG1 response]

SG1 provides P/R and requests move to SG1-OK status: Add Requires clauses for promise (30.6.5 [futures.promise]) `set_exception` (before p18) and `set_exception_at_thread_exit` (before p24): Requires: `p` is not null.

[2015-10, Kona issue prioritization]

Priority 0, move to Ready

Proposed resolution:

This wording is relative to N4431.

Add Requires clauses for promise (30.6.5 [futures.promise]) `set_exception` (before p18) and `set_exception_at_thread_exit` (before p24): *Requires*: `p` is not null.

1. Change 30.6.5 [futures.promise] as depicted:

```
void set_exception(exception_ptr p);
```

??- Requires: `p` is not null.

-18- *Effects*: atomically stores the exception pointer `p` in the shared state and makes that state ready (30.6.4).

```
void set_exception_at_thread_exit(exception_ptr p);
```

??- Requires: `p` is not null.

-24- *Effects*: Stores the exception pointer `p` in the shared state without making that state ready immediately. [...]

2450. (greater|less|greater_equal|less_equal)<void> do not yield a total order for pointers

Section: 20.9.6 [comparisons] **Status:** [Tentatively Ready](#) **Submitter:** Joaquín M López Muñoz **Opened:** 2014-10-30 **Last modified:** 2015-11-04

Priority: 2

View other [active issues](#) in [comparisons].

View all other [issues](#) in [comparisons].

View all issues with [Tentatively Ready](#) status.

Discussion:

`less<void>::operator(t, u)` (and the same applies to the rest of `void` specializations for standard comparison function objects) returns `t < u` even if `t` and `u` are pointers, which by 5.9 [expr.rel]/3 is undefined except if both pointers point to the same array or object. This might be regarded as a specification defect since the intention of N3421 is that `less<>` can substitute for `less<T>` in any case where the latter is applicable. `less<void>` can be rewritten in the following manner to cope with pointers:

```
template<> struct less<void>
{
    typedef unspecified is_transparent;

    template <class T, class U>
    struct pointer_overload : std::is_pointer<std::common_type_t<T, U>>
    {};

    template <
        class T, class U,
        typename std::enable_if<!pointer_overload<T, U>::value>::type* = nullptr
    >
    auto operator()(T&& t, U&& u) const
    -> decltype(std::forward<T>(t) < std::forward<U>(u))
    {
        return std::forward<T>(t) < std::forward<U>(u);
    }

    template <
        class T, class U,
        typename std::enable_if<pointer_overload<T, U>::value>::type* = nullptr
```

```

>
auto operator()(T&& t, U&& u) const
-> decltype(std::declval<std::less<std::common_type_t<T, U>>>()(std::forward<T>(t), std::forward<U>(u)))
{
    std::less<std::common_type_t<T, U>> l;
    return l(std::forward<T>(t), std::forward<U>(u));
}

};

```

This wording is relative to N4140.

1. Change 20.9.6 [comparisons] p14 as indicated:

-14- For templates `greater`, `less`, `greater_equal`, and `less_equal`, the specializations for any pointer type yield a total order, even if the built-in operators `<`, `>`, `<=`, `>=` do not. For template specializations `greater<void>`, `less<void>`, `greater_equal<void>`, and `less_equal<void>`, the call operator with arguments whose common type *CT* is a pointer yields the same value as the corresponding comparison function object class specialization for *CT*.

[2015-02, Cologne]

AM: Is there any way this will be resolved elsewhere? VV: No. AM: Then we should bite the bullet and deal with it here.

MC: These diamond operators are already ugly. Making them more ugly isn't a big problem.

JY found some issue with types that are convertible, and will reword.

Jeffrey suggests improved wording.

[2015-05, Lenexa]

STL: when diamond functions designed, this was on purpose

STL: this does go against the original design

STL: library is smarter and can give a total order

MC: given that the original design rejected this, give back to LEWG

STL: original proposal did not talk about total order

STL: don't feel strongly about changing the design

STL: no objections to taking this issue with some wording changes if people want it

MC: not happy with wording, comparing pointers — what does that mean?

STL: needs careful attention to wording

STL: want to guarantee that `nullptr` participates in total ordering

STL: all hooks into composite pointer type

MC: move from new to open with better wording

STL: to check updates to issue after Lenexa

[2015-06, Telecom]

MC: STL on the hook to update. He's shipping something today so not here.

MC: also add link to [N4229](#)

[2015-10, Kona Saturday afternoon]

STL was on the hook for wording, but STL: I don't care. The architecture on which this is an issue does not exist.

STL: We will also need to incorporate `nullptr`. TK: I think that's implied, since the wording is in terms of the resulting operation, not the deduced types.

STL: Seems legit. MC: I guess I'm OK with this. TK: I'm weakly in favour, so that we can get people to use transparent comparators without worrying.

STL: There's no change to implementations.

Move to Tentatively ready.

Proposed resolution:

This wording is relative to N4296.

1. Change 20.9.6 [comparisons] p14 as indicated:

-14- For templates `greater`, `less`, `greater_equal`, and `less_equal`, the specializations for any pointer type yield a total order, even if the built-in operators `<`, `>`, `<=`, `>=` do not. For template specializations `greater<void>`, `less<void>`, `greater_equal<void>`, and `less_equal<void>`, if the call operator calls a built-in operator comparing pointers, the call operator yields a total order.

2520. N4089 broke initializing `unique_ptr<T[]>` from a `nullptr`

Section: 20.8.1.3.1 [unique.ptr.runtime.ctor] **Status:** [Tentatively Ready](#) **Submitter:** Ville Voutilainen **Opened:** 2015-07-19 **Last modified:** 2015-11-04

Priority: 2

View all issues with [Tentatively Ready](#) status.

Discussion:

According to the wording in 20.8.1.3.1 [unique.ptr.runtime.ctor]/1, this won't work:

```
unique_ptr<int[], DeleterType> x{nullptr, DeleterType{}};
```

`U` is not the same type as pointer, so the first bullet will not do.

`U` is not a pointer type, so the second bullet will not do.

An easy fix would be to add a new bullet after the first bullet, like so:

- `U` is the same type as pointer, or
- `U` is `nullptr_t`, or
- pointer is the same type as `element_type*`, [...]

[2015-10, Kona Saturday afternoon]

MC: Is it the right fix?

GR: It'd be awefully surprising if we had an interface that accepts null pointer values but not `std::nullptr_t`. I think the PR is good.

STL: Are any of the assignments and reset affected? [No, they don't operate on explicit {pointer, deleter} pairs.]

VV: This is already shipping, has been implemented, has been tested and works fine.

Move to Tentatively ready.

Proposed resolution:

This wording is relative to N4527.

1. Change 20.8.1.3.1 [unique.ptr.runtime.ctor] as indicated:

```
template <class U> explicit unique_ptr(U p) noexcept;
template <class U> unique_ptr(U p, see below d) noexcept;
template <class U> unique_ptr(U p, see below d) noexcept;
```

-1- These constructors behave the same as the constructors that take a pointer parameter in the primary template except that they shall not participate in overload resolution unless either

- `U` is the same type as pointer, or
- `U` is `nullptr_t`, or
- pointer is the same type as `element_type*`, `U` is a pointer type `V*`, and `V(*)[]` is convertible to `element_type(*)[]`.

2522. [fund.ts.v2] Contradiction in `set_default_resource` specification

Section: 99 [fund.ts.v2::memory.resource.global] **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2015-07-28 **Last modified:** 2015-11-04

Priority: 2

View all issues with [Tentatively Ready](#) status.

Discussion:

Addresses: [fund.ts.v2](#)

[memory.resource.global]/p7-8 says that the effects of `set_default_resource(r)` are

If `r` is non-null, sets the value of the default memory resource pointer to `r`, otherwise sets the default memory resource pointer to `new_delete_resource()`.

and the operation has the postcondition

```
get_default_resource() == r.
```

When `r` is null, however, the postcondition cannot be met, since the call sets the default memory resource pointer to `new_delete_resource()`, and so `get_default_resource()` would return the value of `new_delete_resource()`, which is obviously not null and so cannot compare equal to `r`.

Previous resolution from Tim Song [SUPERSEDED]:

This wording is relative to [N4480](#).

1. Edit [memory.resource.global]/p8 as follows:

```
-6- memory_resource* set_default_resource(memory_resource* r) noexcept;
```

-7- *Effects*: If `r` is non-null, sets the value of the default memory resource pointer to `r`, otherwise sets the default memory resource pointer to `new_delete_resource()`.

-8- *Postconditions*: `get_default_resource() == r` **if `r` is non-null; otherwise, `get_default_resource() == new_delete_resource()`.**

[...]

[2015-09-15 Geoffrey Romer comments and suggests alternative wording]

Let's just strike 99 [memory.resource.global]/p8. The problem is that p8 is restating p7 incorrectly, but the solution is not to restate p7 correctly, it's to stop trying to restate p7 at all.

[2015-10, Kona Saturday afternoon]

Move to Tentatively ready

[2015-10-26]

Daniel adjusts wording to lib. fund. v2.

Proposed resolution:

This wording is relative to [N4529](#).

1. Edit [memory.resource.global]/p8 as follows:

```
-6- memory_resource* set_default_resource(memory_resource* r) noexcept;
```

-7- *Effects*: If `r` is non-null, sets the value of the default memory resource pointer to `r`, otherwise sets the default memory resource pointer to `new_delete_resource()`.

~~-8- *Postconditions*: `get_default_resource() == r`.~~

[...]

[2523](#). `std::promise` synopsis shows two `set_value_at_thread_exit()`'s for no apparent reason

Section: 30.6.5 [futures.promise] Status: [Ready](#) Submitter: Tim Song Opened: 2015-07-31 Last modified: 2015-11-04

Priority: 0

View other [active issues](#) in [futures.promise].

View all other [issues](#) in [futures.promise].

View all issues with [Ready](#) status.

Discussion:

In 30.6.5 [futures.promise], the class synopsis shows

```
void set_value_at_thread_exit(const R& r);
void set_value_at_thread_exit(see below);
```

There's no apparent reason for having `void set_value_at_thread_exit(const R& r);`, especially as that signature isn't really present in the specializations (particularly `promise<void>`). Note that the similar `set_value` only has a `void set_value(see below);`

While we are here, 30.6.5 [futures.promise]/p1 says that the specializations "differ only in the argument type of the member function `set_value`", which missed `set_value_at_thread_exit`.

[2015-10, Kona issue prioritization]

Priority 0, move to Ready

Proposed resolution:

This wording is relative to N4527.

1. Edit 30.6.5 [futures.promise], class template `promise` synopsis, as indicated:

```
namespace std {
    template <class R>
    class promise {
    public:
        [...]

        // setting the result
        void set_value(see below);
        void set_exception(exception_ptr p);

        // setting the result with deferred notification
        void set_value_at_thread_exit(const R& r);
        void set_value_at_thread_exit(see below);
        void set_exception_at_thread_exit(exception_ptr p);
    };
}
```

2. Edit 30.6.5 [futures.promise]/1 as indicated:

-1- The implementation shall provide the template `promise` and two specializations, `promise<R>` and `promise<void>`. These differ only in the argument type of the member functions `set_value` and `set_value_at_thread_exit`, as set out in ~~its description~~ their descriptions, below.

[2537](#). Constructors for `priority_queue` taking allocators should call `make_heap`

Section: 23.6.5.2 [priqueue.cons.alloc] **Status:** [Ready](#) **Submitter:** Eric Schmidt **Opened:** 2015-09-19 **Last modified:** 2015-11-04

Priority: 0

View all issues with [Ready](#) status.

Discussion:

`priority_queue` constructors taking both `Container` and `Alloc` arguments should finish by calling `make_heap`, just as with the constructors that do not have allocator parameters.

The current reading of 23.6.5.2 [priqueue.cons.alloc], if followed strictly, would effectively require calling code to ensure that the container argument is already a heap, which is probably not what was intended.

[2015-10, Kona issue prioritization]

Priority 0, move to Ready

Proposed resolution:

This wording is relative to N4527.

1. Change 23.6.5.2 [priority_queue.cons.alloc] as indicated:

```
template <class Alloc>
priority_queue(const Compare& compare, const Container& cont, const Alloc& a);
```

-4- *Effects*: Initializes *c* with *cont* as the first argument and *a* as the second argument, and initializes *comp* with *compare*; **calls `make_heap(c.begin(), c.end(), comp)`**.

```
template <class Alloc>
priority_queue(const Compare& compare, Container&& cont, const Alloc& a);
```

-5- *Effects*: Initializes *c* with `std::move(cont)` as the first argument and *a* as the second argument, and initializes *comp* with *compare*; **calls `make_heap(c.begin(), c.end(), comp)`**.

2539. [fund.ts.v2] invocation_trait definition definition doesn't work for surrogate call functions

Section: 99 [fund.ts.v2::meta.trans.other] **Status:** [Tentatively Ready](#) **Submitter:** Mike Spertus **Opened:** 2015-09-25 **Last modified:** 2015-11-04

Priority: Not Prioritized

View all issues with [Tentatively Ready](#) status.

Discussion:

Addresses: fund.ts.v2

In Library Fundamentals 2 (N4529) 3.3.2p3 [meta.trans.other], the definition of invocation traits for a class object *f* considers when *f* is called via a function call operator that is matched by the arguments but ignores the possibility that *f* may be called via a surrogate call function (C++14 13.3.1.1.2 [over.call.object] p2), in which case, the definition of the invocation parameters may be either incorrect or even unsatisfiable.

[2015-10, Kona Saturday afternoon]

AM: Do we have this trait yet? JW: No, it cannot be implemented without compiler support.

Move to tentatively ready

Proposed resolution:

This wording is relative to [N4529](#).

1. In Library Fundamentals 2, change [meta.trans.other] as indicated:

-3- Within this section, define the *invocation parameters* of *INVOKE*(*f*, *t*₁, *t*₂, ..., *t*_N) as follows, in which *t*₁ is the possibly cv-qualified type of *t*₁ and *u*₁ denotes *t*₁& if *t*₁ is an lvalue or *t*₁&& if *t*₁ is an rvalue:

- [...]
- If *f* is a class object, the *invocation parameters* are the parameters matching *t*₁, ..., *t*_N of the best viable function (C++14 §13.3.3) for the arguments *t*₁, ..., *t*_N among the function call operators **and surrogate call functions** of *f*.
- [...]

2545. Simplify wording for bind without explicitly specified return type

Section: 20.9.10.3 [func.bind.bind] **Status:** [Tentatively Ready](#) **Submitter:** Tomasz Kamiński **Opened:** 2015-10-05 **Last modified:** 2016-01-04

Priority: 3

View all other [issues](#) in [func.bind.bind].

View all issues with [Tentatively Ready](#) status.

Discussion:

The specification of the bind overload without return type as of 20.9.10.3 [func.bind.bind] p3, uses the following expression `INVOKE(fd, std::forward<V1>(v1), std::forward<V2>(v2), ..., std::forward<VN>(vN), result_of_t<FD cv & (V1, V2, ..., VN)>)` to describe effects of invocation of returned function.

According to the definition from 20.10.7.6 [meta.trans.other] `result_of_t<FD cv & (V1, V2, ..., VN)>` is equivalent to `decltype(INVOKE(declval<FD cv &>(), declval<V1>(), declval<V2>(), ..., declval<VN>()))`. When combined with the definition of `INVOKE` from 20.9.2 [func.require] p2, the expression `INVOKE(fd, std::forward<V1>(v1), std::forward<V2>(v2), ..., std::forward<VN>(vN), result_of_t<FD cv & (V1, V2, ..., VN)>)` is equivalent to `INVOKE(fd, std::forward<V1>(v1), std::forward<V2>(v2), ..., std::forward<VN>(vN))` implicitly converted to `decltype(INVOKE(declval<FD cv &>(), declval<V1>(), declval<V2>(), ..., declval<VN>()))` (itself).

It is also worth to notice that specifying the result type (R) in `INVOKE(f, args..., R)` does not in any way affect the selected call. As a consequence the use of wording of the form `INVOKE(fd, std::forward<V1>(v1), std::forward<V2>(v2), ..., std::forward<VN>(vN), result_of_t<FD cv & (V1, V2, ..., VN)>)` does not and cannot lead to call of different overload than one invoked by `INVOKE(fd, std::forward<V1>(v1), std::forward<V2>(v2), ..., std::forward<VN>(vN))`.

In summary the form `INVOKE(fd, std::forward<V1>(v1), std::forward<V2>(v2), ..., std::forward<VN>(vN), result_of_t<FD cv & (V1, V2, ..., VN)>)` is a convoluted way of expressing `INVOKE(fd, std::forward<V1>(v1), std::forward<V2>(v2), ..., std::forward<VN>(vN))`, that only confuses reader.

[2015-10, Kona Saturday afternoon]

STL: I most recently reimplemented `std::bind` from scratch, and I think this issue is correct and the solution is good.

Move to Tentatively ready.

Proposed resolution:

This wording is relative to N4527.

1. Change 20.9.10.3 [func.bind.bind] p3 as indicated:

```
template<class F, class... BoundArgs>
unspecified bind(F&& f, BoundArgs&&... bound_args);
```

[...]

-3- *Returns:* A forwarding call wrapper `g` with a weak result type (20.9.2). The effect of `g(u1, u2, ..., uM)` shall be `INVOKE(fd, std::forward<V1>(v1), std::forward<V2>(v2), ..., std::forward<VN>(vN), result_of_t<FD cv & (V1, V2, ..., VN)>)`, where `cv` represents the cv-qualifiers of `g` and the values and types of the bound arguments `v1, v2, ..., vN` are determined as specified below. The copy constructor and move constructor of the forwarding call wrapper shall throw an exception if and only if the corresponding constructor of `FD` or of any of the types `Ti` throws an exception.

[...]

2557. Logical operator traits are broken in the zero-argument case

Section: 20.10.8 [meta.logical] **Status:** [Tentatively Ready](#) **Submitter:** Geoffrey Romer **Opened:** 2015-11-05 **Last modified:** 2016-02-07

Priority: 0

View other [active issues](#) in [meta.logical].

View all other [issues](#) in [meta.logical].

View all issues with [Tentatively Ready](#) status.

Discussion:

The conjunction trait in 20.10.8 [meta.logical] seems intended to support invocation with zero arguments, e.g. `conjunction<>::value`, which is likely to be a useful feature. However, the specification doesn't actually make sense in the zero-argument case. See 20.10.8 [meta.logical]/p3:

The BaseCharacteristic of a specialization `conjunction<B1, ..., BN>` is the first type `B` in the list `true_type, B1, ..., BN` for

which `B::value == false`, or if every `B::value != false` the `BaseCharacteristic` is `BN`.

If "`B1, ..., BN`" is an empty list, then every `B::value != false`, so the `BaseCharacteristic` is `BN`, but there is no `BN` in this case.

(If LWG concludes that `conjunction` intentionally requires at least one argument, I would appreciate their confirmation that I can editorially remove the mention of `true_type`, which seems to have no normative impact outside the zero-argument case.)

Similar comments apply to the `disjunction` trait, and to the corresponding traits in the Fundamentals working paper, see LWG [2558](#).

[2016-02, Issues Telecon]

P0; move to Tentatively Ready.

Proposed resolution:

This wording is relative to N4567.

1. Revise 20.10.8 [meta.logical] as follows:

```
template<class... B> struct conjunction : see below { };
```

[...]

-3- The `BaseCharacteristic` of a specialization `conjunction<B1, ..., BN>` is the first type `Bi` in the list `true_type, B1, ..., BN` for which `Bi::value == false`, or if every `Bi::value != false`, the `BaseCharacteristic` is `BN` **the last type in the list**. [Note: This means a specialization of `conjunction` does not necessarily have a `BaseCharacteristic` of either `true_type` or `false_type`. — end note]

[...]

```
template<class... B> struct disjunction : see below { };
```

[...]

-6- The `BaseCharacteristic` of a specialization `disjunction<B1, ..., BN>` is the first type `Bi` in the list `false_type, B1, ..., BN` for which `Bi::value != false`, or if every `Bi::value == false`, the `BaseCharacteristic` is `BN` **the last type in the list**. [Note: This means a specialization of `disjunction` does not necessarily have a `BaseCharacteristic` of either `true_type` or `false_type`. — end note]

[...]

[2558](#). [fund.ts.v2] Logical operator traits are broken in the zero-argument case

Section: 99 [fund.ts.v2::meta.logical] **Status:** [Tentatively Ready](#) **Submitter:** Geoffrey Romer **Opened:** 2015-11-05 **Last modified:** 2016-02-07

Priority: 0

View other [active issues](#) in [fund.ts.v2::meta.logical].

View all other [issues](#) in [fund.ts.v2::meta.logical].

View all issues with [Tentatively Ready](#) status.

Discussion:

Addresses: [fund.ts.v2](#)

The `conjunction` trait in 99 [meta.logical] seems intended to support invocation with zero arguments, e.g. `conjunction<>::value`, which is likely to be a useful feature. However, the specification doesn't actually make sense in the zero-argument case. See 99 [meta.logical]/p3:

The `BaseCharacteristic` of a specialization `conjunction<B1, ..., BN>` is the first type `B` in the list `true_type, B1, ..., BN` for which `B::value == false`, or if every `B::value != false` the `BaseCharacteristic` is `BN`.

If "`B1, ..., BN`" is an empty list, then every `B::value != false`, so the `BaseCharacteristic` is `BN`, but there is no `BN` in this case.

(If LWG concludes that `conjunction` intentionally requires at least one argument, I would appreciate their confirmation that I can

editorially remove the mention of `true_type`, which seems to have no normative impact outside the zero-argument case.)

Similar comments apply to the `disjunction` trait, and to the corresponding traits in the C++ working paper, see LWG [2557](#).

[2016-02, Issues Telecon]

P0; move to Tentatively Ready.

Proposed resolution:

This wording is relative to [N4562](#).

1. Revise 99 [meta.logical] as follows:

```
template<class... B> struct conjunction : see below { };
```

[...]

-3- The BaseCharacteristic of a specialization `conjunction<B1, ..., BN>` is the first type `Bi` in the list `true_type`, `B1`, ..., `BN` for which `Bi::value == false`, or if every `Bi::value != false`, the BaseCharacteristic is **the last type in the list**. [Note: This means a specialization of `conjunction` does not necessarily have a BaseCharacteristic of either `true_type` or `false_type`. — end note]

[...]

```
template<class... B> struct disjunction : see below { };
```

[...]

-6- The BaseCharacteristic of a specialization `disjunction<B1, ..., BN>` is the first type `Bi` in the list `false_type`, `B1`, ..., `BN` for which `Bi::value != false`, or if every `Bi::value == false`, the BaseCharacteristic is **the last type in the list**. [Note: This means a specialization of `disjunction` does not necessarily have a BaseCharacteristic of either `true_type` or `false_type`. — end note]

[...]

[2559](#). Error in LWG 2234's resolution

Section: 19.3 [assertions] **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2015-11-07 **Last modified:** 2016-02-07

Priority: 0

View other [active issues](#) in [assertions].

View all other [issues](#) in [assertions].

View all issues with [Tentatively Ready](#) status.

Discussion:

The resolution of LWG [2234](#) says that `assert(E)` is a constant subexpression if "`NDEBUG` is defined at the point where `assert(E)` appears".

This is incorrect, as noted in one of STL's comments in that issue's discussion, but was apparently overlooked.

The proposed resolution below just borrows STL's phrasing from the discussion.

[2016-02, Issues Telecon]

P0; move to Tentatively Ready.

Proposed resolution:

This wording is relative to N4567.

1. Change 19.3 [assertions] p2 as indicated:

An expression `assert(E)` is a constant subexpression (17.3.28 [defs.const.subexpr]), if

- `NDEBUG` is defined at the point where ~~`assert(E)` appears~~ **`assert` is last defined or redefined**, or

2560. `is_constructible` underspecified when applied to a function type

Section: 20.10.4.3 [meta.unary.prop] **Status:** [Tentatively Ready](#) **Submitter:** Richard Smith **Opened:** 2015-11-14 **Last modified:** 2016-02-07

Priority: 0

View other [active issues](#) in [meta.unary.prop].

View all other [issues](#) in [meta.unary.prop].

View all issues with [Tentatively Ready](#) status.

Discussion:

What is `is_constructible<void()>::value`? Per 20.10.4.3 [meta.unary.prop] p8:

The predicate condition for a template specialization `is_constructible<T, Args...>` shall be satisfied if and only if the following variable definition would be well-formed for some invented variable `t`:

```
T t(declval<Args>()...);
```

[*Note:* These tokens are never interpreted as a function declaration. — *end note*]

The problem here is that substituting in `τ` as a function type doesn't give a variable definition that's not well-formed (by 1.3.27 [defs.well.formed], well-formed means that it doesn't violate any syntactic or diagnosable semantic rules, and it does not). Instead, it gives a logical absurdity: this wording forces us to imagine a variable of function type, which contradicts the definition of "variable" in 3/6, but does so without violating any diagnosable language rule. So presumably the result must be undefined behavior.

It seems that we need an explicit rule requiring `τ` to be an object or reference type.

Daniel:

As one of the authors of [N3142](#) I would like to express that at least according to my mental model the intention for this trait was to be well-defined for `τ` being a function type with the result of `false` regardless of what the other type arguments are. It would seem like a very unfortunate and unnecessary complication to keep the result as being undefined. First, this result value is symmetric to the result of `is_destructible<T>::value` (where the word covers function types explicitly). Second, if such a resolution would be applied to the working paper, it wouldn't break existing implementations. I have tested clang 3.8.0, gcc 5.x until gcc 6.0, and Visual Studio 2015, all of these implementations evaluate `is_constructible<void()>::value` to `false`.

[2016-02, Issues Telecon]

P0; move to Tentatively Ready.

Proposed resolution:

This wording is relative to N4567.

1. Change 20.10.4.3 [meta.unary.prop], Table 49 — "Type property predicates", as indicated:

Table 49 — Type property predicates

Template	Condition	Preconditions
...		
template <class T, class... Args> struct is_constructible;	For a function type <code>τ</code> , <code>is_constructible<T, Args...>::value</code> is false, otherwise see below	<code>τ</code> and all types in the parameter pack <code>Args</code> shall be complete types, (possibly cv-qualified) void, or arrays of unknown bound.
...		

2565. `std::function`'s move constructor should guarantee nothrow for `reference_wrappers` and function

pointers

Section: 20.9.12.2.1 [func.wrap.func.con] **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2015-12-05 **Last modified:** 2016-02-07

Priority: 0

View all other [issues](#) in [func.wrap.func.con].

View all issues with [Tentatively Ready](#) status.

Discussion:

20.9.12.2.1 [func.wrap.func.con]/5 guarantees that copying a `std::function` whose "target is a callable object passed via `reference_wrapper` or a function pointer" does not throw exceptions, but the standard doesn't provide this guarantee for the move constructor, which makes scant sense.

[2016-02, Issues Telecon]

P0; move to Tentatively Ready.

Proposed resolution:

This wording is relative to N4567.

[Drafting note: The inserted paragraph is a copy of 20.9.12.2.1 [func.wrap.func.con]/5, only changing "copy constructor" to "copy or move constructor". It does not attempt to fix the issue identified in LWG [2370](#), whose P/R will likely need updating if this wording is adopted.]

1. Insert after 20.9.12.2.1 [func.wrap.func.con]/6:

```
function(function&& f);
template <class A> function(allocator_arg_t, const A& a, function&& f);
```

-6- *Effects:* If !f, *this has no target; otherwise, move-constructs the target of f into the target of *this, leaving f in a valid state with an unspecified value.

-?- *Throws:* Shall not throw exceptions if f's target is a callable object passed via `reference_wrapper` or a function pointer. Otherwise, may throw `bad_alloc` or any exception thrown by the copy or move constructor of the stored callable object. [Note: Implementations are encouraged to avoid the use of dynamically allocated memory for small callable objects, for example, where f's target is an object holding only a pointer or reference to an object and a member function pointer. — end note]

[2566](#). Requirements on the first template parameter of container adaptors

Section: 23.6 [container.adaptors] **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2015-12-08 **Last modified:** 2016-02-07

Priority: 0

View all other [issues](#) in [container.adaptors].

View all issues with [Tentatively Ready](#) status.

Discussion:

As noted in [this StackOverflow question](#), 23.6 [container.adaptors] doesn't seem to place any requirement on the first template parameter (τ) of `stack`, `queue`, and `priority_queue`: the only use of τ is in the default template argument (which need not be used) for the second template parameter (`Container`), while all of the operations of the adaptors are defined using `Container`'s member typedefs.

This permits confusing and arguably nonsensical types like `queue<double, deque<std::string>>` or `priority_queue<std::nothrow_t, vector<int>>`, which presumably wasn't intended.

[2016-02, Issues Telecon]

P0; move to Tentatively Ready.

Proposed resolution:

This wording is relative to N4567.

1. Edit 23.6.1 [container.adaptors.general]/2 as indicated:

-2- The container adaptors each take a `Container` template parameter, and each constructor takes a `Container` reference argument. This container is copied into the `Container` member of each adaptor. If the container takes an allocator, then a compatible allocator may be passed in to the adaptor's constructor. Otherwise, normal copy or move construction is used for the container argument. The first template parameter τ of the container adaptors shall denote the same type as `Container::value_type`.

2571. §[map.modifiers]/2 imposes nonsensical requirement on `insert(InputIterator, InputIterator)`

Section: 23.4.4.4 [map.modifiers] **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2015-12-12 **Last modified:** 2016-02-12

Priority: 0

View all other [issues](#) in [map.modifiers].

View all issues with [Tentatively Ready](#) status.

Discussion:

The initial paragraphs of 23.4.4.4 [map.modifiers] currently read:

```
template <class P> pair<iterator, bool> insert(P&& x);
template <class P> iterator insert(const_iterator position, P&& x);
template <class InputIterator>
void insert(InputIterator first, InputIterator last);
```

-1- *Effects:* The first form is equivalent to `return emplace(std::forward<P>(x))`. The second form is equivalent to `return emplace_hint(position, std::forward<P>(x))`.

-2- *Remarks:* These signatures shall not participate in overload resolution unless `std::is_constructible<value_type, P&&>::value` is true.

Clearly, p2's requirement makes no sense for `insert(InputIterator, InputIterator)` - it doesn't even have a template parameter called `P`.

This paragraph used to have text saying "The signature taking `InputIterator` parameters does not require `CopyConstructible` of either `key_type` or `mapped_type` if the dereferenced `InputIterator` returns a non-const rvalue `pair<key_type, mapped_type>`. Otherwise `CopyConstructible` is required for both `key_type` and `mapped_type`", but that was removed by LWG [2005](#), whose PR was written as if that overload didn't exist in the text.

It looks like the text addressing this overload is redundant to the requirements on `a.insert(i, j)` in Table 102 that `value_type` be `EmplaceConstructible` from `*i`. If so, then the signature should just be deleted from this section.

[2016-02, Issues Telecon]

P0; move to Tentatively Ready.

Proposed resolution:

This wording is relative to N4567.

1. Edit 23.4.4.4 [map.modifiers] as indicated:

```
template <class P> pair<iterator, bool> insert(P&& x);
template <class P> iterator insert(const_iterator position, P&& x);
template <class InputIterator>
void insert(InputIterator first, InputIterator last);
```

-1- *Effects:* The first form is equivalent to `return emplace(std::forward<P>(x))`. The second form is equivalent to `return emplace_hint(position, std::forward<P>(x))`.

-2- *Remarks:* These signatures shall not participate in overload resolution unless `std::is_constructible<value_type, P&&>::value` is true.

[2572](#). The remarks for `shared_ptr::operator*` should apply to cv-qualified void as well

Section: 20.8.2.2.5 [util.smartptr.shared.obs] **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2015-12-13 **Last modified:** 2016-02-07

Priority: 0

View other [active issues](#) in [util.smartptr.shared.obs].

View all other [issues](#) in [util.smartptr.shared.obs].

View all issues with [Tentatively Ready](#) status.

Discussion:

20.8.2.2.5 [util.smartptr.shared.obs]/4 says for `shared_ptr::operator*`

Remarks: When τ is void, it is unspecified whether this member function is declared. If it is declared, it is unspecified what its return type is, except that the declaration (although not necessarily the definition) of the function shall be well formed.

This remark should also apply when τ is cv-qualified void (compare LWG [2500](#)).

[2016-02, Issues Telecon]

P0; move to Tentatively Ready.

Proposed resolution:

This wording is relative to N4567.

1. Edit 20.8.2.2.5 [util.smartptr.shared.obs]/4 as indicated:

```
T& operator*() const noexcept;
```

```
[...]
```

-4- *Remarks:* When τ is [\(possibly cv-qualified\)](#) void, it is unspecified whether this member function is declared. If it is declared, it is unspecified what its return type is, except that the declaration (although not necessarily the definition) of the function shall be well formed.

[2574](#). `[fund.ts.v2] std::experimental::function::operator=(F&&)` should be constrained

Section: 99 [fund.ts.v2::func.wrap.func.con] **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2015-12-05 **Last modified:** 2016-02-07

Priority: 0

View other [active issues](#) in [fund.ts.v2::func.wrap.func.con].

View all other [issues](#) in [fund.ts.v2::func.wrap.func.con].

View all issues with [Tentatively Ready](#) status.

Discussion:

Addresses: fund.ts.v2

LWG [2132](#) constrained `std::function`'s constructor and assignment operator from callable objects for C++14. The constructors of `std::experimental::function` isn't separately specified in the fundamentals TS and so inherited the constraints from C++14, but the assignment operator is separately specified and presumably needs to be constrained.

[2016-02, Issues Telecon]

P0; move to Tentatively Ready.

Proposed resolution:

This wording is relative to [N4562](#).

1. Insert a paragraph after 99 [func.wrap.func.con]/15 as indicated:

```
template<class F> function& operator=(F&& f);
```

-14- *Effects*: function(allocator_arg, ALLOCATOR_OF(*this), std::forward<F>(f)).swap(*this);

-15- *Returns*: *this.

-?- *Remarks*: This assignment operator shall not participate in overload resolution unless `declval<decay_t<F>&>()` is Callable (C++14 §20.9.11.2) for argument types `ArgTypes...` and return type `R`.

2575. [fund.ts.v2] experimental::function::assign should be removed

Section: 99 [fund.ts.v2::func.wrap.func] **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2015-12-20 **Last modified:** 2016-02-07

Priority: 0

View other [active issues](#) in [fund.ts.v2::func.wrap.func].

View all other [issues](#) in [fund.ts.v2::func.wrap.func].

View all issues with [Tentatively Ready](#) status.

Discussion:

Addresses: fund.ts.v2

Following the lead of LWG [2385](#), the `assign(F&&, const A&)` member function template in `std::experimental::function` makes no sense (it causes undefined behavior unless the allocator passed compares equal to the one already used by `*this`) and should be removed.

[2016-02, Issues Telecon]

P0; move to Tentatively Ready.

Proposed resolution:

This wording is relative to [N4562](#).

1. Edit 99 [func.wrap.func], class template function synopsis, as indicated:

```
namespace std {
  namespace experimental {
    inline namespace fundamentals_v2 {
      [...]
      template<class R, class... ArgTypes>
      class function<R(ArgTypes...)> {
      public:
        [...]
        void swap(function&);
        template<class F, class A> void assign(F&&, const A&);
        [...]
      };
      [...]
    } // namespace fundamentals_v2
  } // namespace experimental
  [...]
} // namespace std
```

2576. istream_iterator and ostream_iterator should use std::addressof

Section: 24.6 [stream.iterators] **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2016-01-01 **Last modified:** 2016-02-07

Priority: 0

View all other [issues](#) in [stream.iterators].

View all issues with [Tentatively Ready](#) status.

Discussion:

To defend against overloaded unary &. This includes the constructors of both iterators, and `istream_iterator::operator->`.

Note that `{i,o}stream_type` are specializations of `basic_{i,o}stream`, but the constructors might still pick up an overloaded & via the `traits` template parameter. This change also provides consistency with `std::experimental::ostream_joiner` (which uses `std::addressof`).

[2016-02, Issues Telecon]

P0; move to Tentatively Ready.

Proposed resolution:

This wording is relative to N4567.

1. Edit 24.6.1.1 [istream.iterator.cons]/3+4 as indicated:

```
istream_iterator(istream_type& s);
```

-3- *Effects*: Initializes `in_stream` with `&saddressof(s)`. value may be initialized during construction or the first time it is referenced.

-4- *Postcondition*: `in_stream == &saddressof(s)`.

2. Edit 24.6.1.2 [istream.iterator.ops]/2 as indicated:

```
const T* operator->() const;
```

-2- *Returns*: `&saddressof(operator*())`.

3. Edit 24.6.2.1 [ostream.iterator.cons.des]/1+2 as indicated:

```
ostream_iterator(ostream_type& s);
```

-1- *Effects*: Initializes `out_stream` with `&saddressof(s)` and `delim` with `null`.

```
ostream_iterator(ostream_type& s, const charT* delimiter);
```

-2- *Effects*: Initializes `out_stream` with `&saddressof(s)` and `delim` with `delimiter`.

2577. {shared,unique}_lock should use std::addressof

Section: 30.4.2.2.1 [thread.lock.unique.cons], 30.4.2.3.1 [thread.lock.shared.cons] **Status:** [Tentatively Ready](#) **Submitter:** Tim Song
Opened: 2016-01-01 **Last modified:** 2016-02-07

Priority: 0

View all other [issues](#) in [thread.lock.unique.cons].

View all issues with [Tentatively Ready](#) status.

Discussion:

So that they work with user-defined types that have overloaded unary &.

[2016-02, Issues Telecon]

P0; move to Tentatively Ready.

Proposed resolution:

This wording is relative to N4567.

1. Edit 30.4.2.2.1 [thread.lock.unique.cons] as indicated:

```
explicit unique_lock(mutex_type& m);
```

[...]

-5- *Postconditions*: `pm == &maddressof(m)` and `owns == true`.

```
unique_lock(mutex_type& m, defer_lock_t) noexcept;
```

[...]

-7- *Postconditions*: `pm == &addressof(m)` and `owns == false`.

```
unique_lock(mutex_type& m, try_to_lock_t);
```

[...]

-10- *Postconditions*: `pm == &addressof(m)` and `owns == res`, where `res` is the value returned by the call to `m.try_lock()`.

```
unique_lock(mutex_type& m, adopt_lock_t);
```

[...]

-13- *Postconditions*: `pm == &addressof(m)` and `owns == true`.

-14- *Throws*: Nothing.

```
template <class Clock, class Duration>
unique_lock(mutex_type& m, const chrono::time_point<Clock, Duration>& abs_time);
```

[...]

-17- *Postconditions*: `pm == &addressof(m)` and `owns == res`, where `res` is the value returned by the call to `m.try_lock_until(abs_time)`.

```
template <class Rep, class Period>
unique_lock(mutex_type& m, const chrono::duration<Rep, Period>& rel_time);
```

[...]

-20- *Postconditions*: `pm == &addressof(m)` and `owns == res`, where `res` is the value returned by the call to `m.try_lock_for(rel_time)`.

2. Edit 30.4.2.3.1 [thread.lock.shared.cons] as indicated:

```
explicit shared_lock(mutex_type& m);
```

[...]

-5- *Postconditions*: `pm == &addressof(m)` and `owns == true`.

```
shared_lock(mutex_type& m, defer_lock_t) noexcept;
```

[...]

-7- *Postconditions*: `pm == &addressof(m)` and `owns == false`.

```
shared_lock(mutex_type& m, try_to_lock_t);
```

[...]

-10- *Postconditions*: `pm == &addressof(m)` and `owns == res` where `res` is the value returned by the call to `m.try_lock_shared()`.

```
shared_lock(mutex_type& m, adopt_lock_t);
```

[...]

-13- *Postconditions*: `pm == &addressof(m)` and `owns == true`.

```
template <class Clock, class Duration>
shared_lock(mutex_type& m,
            const chrono::time_point<Clock, Duration>& abs_time);
```

[...]

-16- *Postconditions*: `pm == &addressof(m)` and `owns == res` where `res` is the value returned by the call to `m.try_lock_shared_until(abs_time)`.

```
template <class Rep, class Period>
shared_lock(mutex_type& m,
            const chrono::duration<Rep, Period>& rel_time);
```

[...]

-19- *Postconditions*: `pm == &addressof(m)` and `owns == res` where `res` is the value returned by the call to `m.try_lock_shared_for(rel_time)`.

2579. Inconsistency wrt Allocators in `basic_string` assignment vs. `basic_string::assign`

Section: 21.4.6.3 [`string::assign`] **Status:** [Tentatively Ready](#) **Submitter:** Marshall Clow **Opened:** 2016-01-05 **Last modified:** 2016-02-07

Priority: 0

View all issues with [Tentatively Ready](#) status.

Discussion:

In issue [2063](#), we changed the *Effects* of `basic_string::assign(basic_string&&)` to match the behavior of `basic_string::operator=(basic_string&&)`, making them consistent.

We did not consider `basic_string::assign(const basic_string&)`, and its *Effects* differ from `operator=(const basic_string&)`.

Given the following definition:

```
typedef std::basic_string<char, std::char_traits<char>, MyAllocator<char>> MyString;

MyAllocator<char> alloc1, alloc2;
MyString string1("Alloc1", alloc1);
MyString string2(alloc2);
```

the following bits of code are not equivalent:

```
string2 = string1;           // (a) calls operator=(const MyString&)
string2.assign(string1);     // (b) calls MyString::assign(const MyString&)
```

What is the allocator for `string2` after each of these calls?

- If `MyAllocator<char>::propagate_on_container_copy_assignment` is true, then it should be `alloc2`, otherwise it should be `alloc1`.
- `alloc2`

21.4.6.3 [`string::assign`]/1 says that (b) is equivalent to `assign(string1, 0, npos)`, which eventually calls `assign(str.data() + pos, rlen)`. No allocator transfer there.

[2016-02, Issues Telecon]

P0; move to Tentatively Ready.

Proposed resolution:

This wording is relative to N4567.

- Modify 21.4.6.3 [`string::assign`] p.1 as indicated:

```
basic_string& assign(const basic_string& str);
```

-1- *Effects*: Equivalent to `*this = str` `assign(str, 0, npos)`.

-2- *Returns*: `*this`.

2581. Specialization of `<type_traits>` variable templates should be prohibited

Section: 20.10.2 [`meta.type.synop`] **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2016-01-07 **Last modified:** 2016-02-07

Priority: 0

View all issues with [Tentatively Ready](#) status.

Discussion:

20.10.2 [meta.type.synop]/1 only prohibits adding specializations of class templates in <type_traits>. Now that we have _v variable templates, this prohibition should apply to them as well.

[2016-02, Issues Telecon]

P0; move to Tentatively Ready.

Proposed resolution:

This wording is relative to N4567.

1. Edit 20.10.2 [meta.type.synop]/1 as indicated:

-1- The behavior of a program that adds specializations for any of the **class** templates defined in this subclause is undefined unless otherwise specified.

[2582](#). [§\[res.on.functions\]/2](#)'s prohibition against incomplete types shouldn't apply to type traits

Section: 20.10 [meta] **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2016-01-07 **Last modified:** 2016-02-12

Priority: 0

View other [active issues](#) in [meta].

View all other [issues](#) in [meta].

View all issues with [Tentatively Ready](#) status.

Discussion:

17.6.4.8 [res.on.functions]/2.5 says that the behavior is undefined "if an incomplete type is used as a template argument when instantiating a template component, unless specifically allowed for that component."

This rule should not apply to type traits — a literal application would make `is_same<void, void>` undefined behavior, since nothing in 20.10 [meta] (or elsewhere) "specifically allows" instantiating `is_same` with incomplete types.

Traits that require complete types are already explicitly specified as such, so the proposed wording below simply negates 17.6.4.8 [res.on.functions]/2.5 for 20.10 [meta].

[2016-02, Issues Telecon]

P0; move to Tentatively Ready.

Proposed resolution:

This wording is relative to N4567.

1. Insert a new paragraph after 20.10.2 [meta.type.synop]/1:

?- Unless otherwise specified, an incomplete type may be used to instantiate a template in this subclause.

[2583](#). There is no way to supply an allocator for `basic_string(str, pos)`

Section: 21.4.2 [string.cons] **Status:** [Tentatively Ready](#) **Submitter:** Pablo Halpern **Opened:** 2016-01-05 **Last modified:** 2016-02-07

Priority: 0

View other [active issues](#) in [string.cons].

View all other [issues](#) in [string.cons].

View all issues with [Tentatively Ready](#) status.

Discussion:

Container and `string` constructors in the standard follow two general rules:

1. Every constructor needs a version with and without an allocator argument (possibly through the use of default arguments).

2. Every constructor *except the copy constructor* for which an allocator is not provided uses a default-constructed allocator.

The first rule ensures emplacing a string into a container that uses a scoped allocator will correctly propagate the container's allocator to the new element.

The current standard allows constructing a string as `basic_string(str, pos)` but not `basic_string(str, pos, alloc)`. This omission breaks the first rule and causes something like the following to fail:

```
typedef basic_string<char, char_traits<char>, A<char>> stringA;
vector<stringA, scoped_allocator_adaptor<A<stringA>>> vs;
stringA s;

vs.emplace_back(s, 2); // Ill-formed
```

[2016-02, Issues Telecon]

P0; move to Tentatively Ready.

Proposed resolution:

This wording is relative to N4567.

- 1. Change 21.4 [basic.string], class template `basic_string` synopsis, as indicated

```
basic_string(const basic_string& str, size_type pos, size_type n = npos,
             const Allocator& a = Allocator());
basic_string(const basic_string& str, size_type pos, size_type n,
             const Allocator& a = Allocator());
```

- 2. Change 21.4.2 [string.cons] as indicated

```
basic_string(const basic_string& str,
             size_type pos, size_type n = npos,
             const Allocator& a = Allocator());
```

-3- *Throws*: `out_of_range` if `pos > str.size()`.

-4- *Effects*: Constructs an object of class `basic_string` and determines the effective length `rlen` of the initial string value as the smaller of `n` and `str.size() - pos`, as indicated in Table 65.

```
basic_string(const basic_string& str, size_type pos, size_type n,
             const Allocator& a = Allocator());
```

-?- *Throws*: `out_of_range` if `pos > str.size()`.

-?- *Effects*: Constructs an object of class `basic_string` and determines the effective length `rlen` of the initial string value as the smaller of `n` and `str.size() - pos`, as indicated in Table 65.

Table 65 — `basic_string(const basic_string&, size_type, size_type, const Allocator&)` and `basic_string(const basic_string&, size_type, size_type, const Allocator&)` effects

Element	Value
<code>data()</code>	points at the first element of an allocated copy of <code>rlen</code> consecutive elements of the string controlled by <code>str</code> beginning at position <code>pos</code>
<code>size()</code>	<code>rlen</code>
<code>capacity()</code>	a value at least as large as <code>size()</code>

2585. `forward_list::resize(size_type, const value_type&)` effects incorrect

Section: 23.3.9.5 [forwardlist.modifiers] **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2016-01-14 **Last modified:** 2016-02-07

Priority: 0

View all other [issues](#) in [forwardlist.modifiers].

View all issues with [Tentatively Ready](#) status.

Discussion:

23.3.9.5 [forwardlist.modifiers]/29 says that the effects of `forward_list::resize(size_type sz, const value_type& c)` are:

Effects: If `sz < distance(begin(), end())`, erases the last `distance(begin(), end()) - sz` elements from the list. Otherwise, inserts `sz - distance(begin(), end())` elements at the end of the list such that each new element, `e`, is initialized by a method equivalent to calling `allocator_traits<allocator_type>::construct(get_allocator(), std::addressof(e), c)`.

In light of LWG [2218](#), the use of `allocator_traits<allocator_type>::construct` is incorrect, as a rebound allocator may be used. There's no need to repeat this information, in any event — no other specification of `resize()` does it.

[2016-02, Issues Telecon]

P0; move to Tentatively Ready.

Proposed resolution:

This wording is relative to N4567.

1. Edit 23.3.9.5 [forwardlist.modifiers]/29 as indicated:

[Drafting note: "copies of `c`" is the phrase used by `vector::resize` and `deque::resize`.]

```
void resize(size_type sz, const value_type& c);
```

-29- *Effects:* If `sz < distance(begin(), end())`, erases the last `distance(begin(), end()) - sz` elements from the list. Otherwise, inserts `sz - distance(begin(), end())` **elements** **copies of `c`** at the end of the list ~~such that each new element, `e`, is initialized by a method equivalent to calling `allocator_traits<allocator_type>::construct(get_allocator(), std::addressof(e), c)`.~~

[2586](#). Wrong value category used in `scoped_allocator_adaptor::construct()`

Section: 20.13.4 [allocator.adaptor.members], 20.7.7.2 [allocator.uses.construction] **Status:** [Tentatively Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2016-01-15 **Last modified:** 2016-02-07

Priority: 0

View other [active issues](#) in [allocator.adaptor.members].

View all other [issues](#) in [allocator.adaptor.members].

View all issues with [Tentatively Ready](#) status.

Discussion:

20.13.4 [allocator.adaptor.members] p9 says that the `is_constructible` tests are done using `inner_allocator_type`, which checks for construction from an rvalue, but then the constructor is passed `inner_allocator()` which returns a non-const lvalue reference. The value categories should be consistent, otherwise this fails to compile:

```
#include <memory>
#include <scoped_allocator>

struct X {
    using allocator_type = std::allocator<X>;
    X(std::allocator_arg_t, allocator_type&&) { }
    X(allocator_type&) { }
};

int main() {
    std::scoped_allocator_adaptor<std::allocator<X>> sa;
    sa.construct(sa.allocate(1));
}
```

`uses_allocator<X, decltype(sa)::inner_allocator_type>>` is true, because it can be constructed from an rvalue of the allocator type, so bullet (9.1) doesn't apply.

`is_constructible<X, allocator_arg_t, decltype(sa)::inner_allocator_type>` is true, so bullet (9.2) applies. That means we try to construct the object passing it `sa.inner_allocator()` which is an lvalue reference, so it fails.

The `is_constructible` checks should use an lvalue reference, as that's what's actually going to be used.

I don't think the same problem exists in the related wording in 20.7.7.2 [allocator.uses.construction] if we assume that the value categories of `v1`, `v2`, ..., `vN` and `alloc` are meant to be preserved, so that the `is_constructible` traits and the initialization expressions match. However, it does say "an allocator `alloc` of type `Alloc`" and if `Alloc` is an reference type then it's not an allocator, so I suggest a small tweak there too.

[2016-02, Issues Telecon]

Strike first paragraph of PR, and move to Tentatively Ready.

Original Resolution [SUPERSEDED]:

1. Change 20.7.7.2 [allocator.uses.construction] p1:

-1- Uses-allocator construction with allocator `Alloc` refers to the construction of an object `obj` of type `T`, using constructor arguments `v1`, `v2`, ..., `vN` of types `V1`, `V2`, ..., `VN`, respectively, and an allocator [\(or reference to an allocator\)](#) `alloc` of type `Alloc`, according to the following rules:

2. Change the 2nd and 3rd bullets in 20.13.4 [allocator.adaptor.members] p9 to add two lvalue-references:

(9.2) — Otherwise, if `uses_allocator<T, inner_allocator_type>::value` is true and `is_constructible<T, allocator_arg_t, inner_allocator_type&, Args...>::value` is true, calls `OUTERMOST_ALLOC_TRAITS(*this)::construct(OUTERMOST(*this), p, allocator_arg, inner_allocator(), std::forward<Args>(args)...)...`.

(9.3) — Otherwise, if `uses_allocator<T, inner_allocator_type>::value` is true and `is_constructible<T, Args..., inner_allocator_type&>::value` is true, calls `OUTERMOST_ALLOC_TRAITS(*this)::construct(OUTERMOST(*this), p, std::forward<Args>(args)..., inner_allocator())`.

3. Change the 2nd, 3rd, 6th, and 7th bullets in 20.13.4 [allocator.adaptor.members] p11 to add four lvalue-references:

(11.2) — Otherwise, if `uses_allocator<T1, inner_allocator_type>::value` is true and `is_constructible<T1, allocator_arg_t, inner_allocator_type&, Args1...>::value` is true, then `xprime` is `tuple_cat(tuple<allocator_arg_t, inner_allocator_type&>(allocator_arg, inner_allocator()), std::move(x))`.

(11.3) — Otherwise, if `uses_allocator<T1, inner_allocator_type>::value` is true and `is_constructible<T1, Args1..., inner_allocator_type&>::value` is true, then `xprime` is `tuple_cat(std::move(x), tuple<inner_allocator_type&>(inner_allocator()))`.

[...]

(11.6) — Otherwise, if `uses_allocator<T2, inner_allocator_type>::value` is true and `is_constructible<T2, allocator_arg_t, inner_allocator_type&, Args2...>::value` is true, then `yprime` is `tuple_cat(tuple<allocator_arg_t, inner_allocator_type&>(allocator_arg, inner_allocator()), std::move(y))`.

(11.7) — Otherwise, if `uses_allocator<T2, inner_allocator_type>::value` is true and `is_constructible<T2, Args2..., inner_allocator_type&>::value` is true, then `yprime` is `tuple_cat(std::move(y), tuple<inner_allocator_type&>(inner_allocator()))`.

[2016-02, Issues Telecon]

P0; move to Tentatively Ready.

Proposed resolution:

This wording is relative to N4567.

1. Change the 2nd and 3rd bullets in 20.13.4 [allocator.adaptor.members] p9 to add two lvalue-references:

(9.2) — Otherwise, if `uses_allocator<T, inner_allocator_type>::value` is true and `is_constructible<T, allocator_arg_t, inner_allocator_type&, Args...>::value` is true, calls `OUTERMOST_ALLOC_TRAITS(*this)::construct(OUTERMOST(*this), p, allocator_arg, inner_allocator(), std::forward<Args>(args)...)...`.

(9.3) — Otherwise, if `uses_allocator<T, inner_allocator_type>::value` is true and `is_constructible<T, Args..., inner_allocator_type>::value` is true, calls `OUTERMOST_ALLOC_TRAITS(*this)::construct(OUTERMOST(*this), p, std::forward<Args>(args)..., inner_allocator())`.

2. Change the 2nd, 3rd, 6th, and 7th bullets in 20.13.4 [allocator.adaptor.members] p11 to add four lvalue-references:

(11.2) — Otherwise, if `uses_allocator<T1, inner_allocator_type>::value` is true and `is_constructible<T1, allocator_arg_t, inner_allocator_type, Args1...>::value` is true, then `xprime` is `tuple_cat(tuple<allocator_arg_t, inner_allocator_type>(allocator_arg, inner_allocator()), std::move(x))`.

(11.3) — Otherwise, if `uses_allocator<T1, inner_allocator_type>::value` is true and `is_constructible<T1, Args1..., inner_allocator_type>::value` is true, then `xprime` is `tuple_cat(std::move(x), tuple<inner_allocator_type>(inner_allocator()))`.

[...]

(11.6) — Otherwise, if `uses_allocator<T2, inner_allocator_type>::value` is true and `is_constructible<T2, allocator_arg_t, inner_allocator_type, Args2...>::value` is true, then `yprime` is `tuple_cat(tuple<allocator_arg_t, inner_allocator_type>(allocator_arg, inner_allocator()), std::move(y))`.

(11.7) — Otherwise, if `uses_allocator<T2, inner_allocator_type>::value` is true and `is_constructible<T2, Args2..., inner_allocator_type>::value` is true, then `yprime` is `tuple_cat(std::move(y), tuple<inner_allocator_type>(inner_allocator()))`.

2590. Aggregate initialization for `std::array`

Section: 23.3.7.1 [array.overview] **Status:** [Tentatively Ready](#) **Submitter:** Robert Haberlach **Opened:** 2016-01-30 **Last modified:** 2016-02-07

Priority: 0

View other [active issues](#) in [array.overview].

View all other [issues](#) in [array.overview].

View all issues with [Tentatively Ready](#) status.

Discussion:

Similar to [core issue 1270's](#) resolution, 23.3.7.1 [array.overview]/2 should cover aggregate-initialization in general. As it stands, that paragraph solely mentions copy-list-initialization — i.e. it is unclear whether the following notation is (guaranteed to be) well-formed:

```
std::array<int, 1> arr{0};
```

[2016-02, Issues Telecon]

P0; move to Tentatively Ready.

Proposed resolution:

This wording is relative to N4567.

1. Change 23.3.7.1 [array.overview] p2 as indicated:

-2- An array is an aggregate (8.5.1) that can be [list](#)-initialized with [the syntax](#)

```
array<T, N> a = { initializer-list };
```

where *initializer-list* is a comma-separated list of up to *N* elements whose types are convertible to *T*.